

archeobooks_ingest: a manual

Adding books to the archaeoastronomy library database from photographs, and keeping the database honest.

1. What the program does

The library's website at <https://bpenprase.github.io/archeobooks/> is a small browser that reads a single SQLite file, `archeo2.db`, which lives in the GitHub repository and holds one row per book: title, author, publisher, publication year, keywords, page count, and a multi-paragraph summary. `archeobooks_ingest.py` is the tool for adding rows to that file and for repairing rows that are already there.

Given a folder of phone photographs of book covers, title pages or copyright pages, it reads the bibliographic details off the pages, merges the photographs that belong to the same book, checks each book against the database so that nothing is entered twice, and then, for each new book, carries out two calls to Claude: a research call, with web search switched on, that verifies the publisher, year and page count and gathers what the book actually contains from publisher pages, library catalogues and reviews; and a writing call, with no tools, that composes the summary from those verified notes under strict instructions never to invent quotations, page numbers or chapter structures. The result is a new database file, written beside the original, and a report that shows every entry together with a confidence rating, the sources it rests on, and the research notes it was written from, so that you can judge the quality before anything reaches the website.

The same machinery can regenerate entries that are already in the database, either by id or by selecting the ones that show the failure modes of the older pipeline, and it can remove duplicate rows. Long runs write each finished entry immediately and keep a checkpoint, so they can be interrupted and resumed.

Two companion scripts help you judge the results: `preview_db.py` opens any database file in the library's own web interface, running locally, so that you can browse it exactly as students will, and `compare_entries.py` puts the same books from two or three database files side by side in one document. A third, `make_labels.py`, turns the shelf codes into printable spine labels.

2. Files and folders

Everything lives in one working folder, which this manual assumes is called `archeobooks_ingest`:

<code>archeobooks_ingest/</code>	
<code>archeobooks_ingest.py</code>	the program
<code>preview_db.py</code>	the previewer
<code>compare_entries.py</code>	side-by-side comparison of two or three database files
<code>make_labels.py</code>	Avery 5160 shelf labels from a spreadsheet of codes
<code>README.md</code> , <code>MANUAL.md</code> , <code>PREVIEW_MANUAL.md</code>	documentation
<code>venv/</code>	the private Python environment, created once
<code>test_images/</code>	a folder of photographs (any name; you may have several)
<code>ingest_work/</code>	everything the program produces

Inside `ingest_work` you will find, after a run:

File	What it is
archeo2_current.db	The pristine copy downloaded from GitHub at the start of a fresh run. Never modified.
labels_<date>.csv	Code, Title, Author, Publisher and Year for the entries that run added or replaced, ready for <code>make_labels.py</code> .
archeo2_new_<date>.db	The output of a run: the source database plus that run's additions, replacements and deletions. One per run.
ingest_report_<date>.md	The human-readable report for that run: a summary table, then every entry in full with its confidence, verification notes, sources and research notes.
ingest_results_<date>.json	The same information in machine-readable form.
candidates.json	What was read from the photographs, merged into one record per book. Editable.
extractions_cache.json	The raw reading of every photograph the program has ever seen, so that photographs are never sent twice.
checkpoint.json	Progress of the most recent run, used by <code>--resume</code> .

The `<date>` stamp has the form `20260908_161506` (year, month, day, hour, minute, second). All the files from one run share the same stamp.

3. Setting up, and starting each session

Once. Install Python 3 if `python3 --version` in Terminal does not report a version. Put the two scripts in the working folder, then in Terminal:

```
cd /path/to/archeobooks_ingest
python3 -m venv venv
source venv/bin/activate
python3 -m pip install anthropic pillow
```

Create an API key at console.anthropic.com. If your photographs come straight from an iPhone as `.heic` files, also run `python3 -m pip install pillow-heif`; ordinary JPGs need nothing extra.

Every time you open a new Terminal window, three lines bring you back to a working state:

```
cd /path/to/archeobooks_ingest
source venv/bin/activate
export ANTHROPIC_API_KEY="sk-ant-..."
```

(Typing `cd` and then dragging the folder from the Finder onto the Terminal window pastes its path.) You should see `(venv)` at the start of the prompt; if you do not, the second line has not been run.

4. Command reference

```
python3 archeobooks_ingest.py [what to do] [where to start from] [options]
```

What to do (at least one):

Flag	Meaning
<code>--images FOLDER</code>	Read the photographs in FOLDER and add the books they show.
<code>--manifest FILE</code>	Use a previously written (and possibly hand-corrected) <code>candidates.json</code> instead of reading photographs.
<code>--refresh IDS</code>	Regenerate existing entries. IDS may be a list and ranges (<code>167,878,1-50</code>), the word <code>suspect</code> (entries that are blank, that begin mid-stream with "Chapter ...", that quote page numbers, or that are very short), or the word <code>all</code> .
<code>--dedupe</code>	Remove rows whose titles are identical once capitalisation and punctuation are ignored, keeping the row with the longest summary.
<code>--resume</code>	Continue the most recent interrupted run from its checkpoint.
<code>--add-codes</code>	Add the <code>Code</code> column and give every entry a shelf code. Makes no AI calls and costs nothing.
<code>--scrub FILE.db</code>	Only strip stray citation markup from the summaries in FILE.db, in place, then stop.

Where to start from:

Flag	Meaning
<i>(nothing)</i>	Download the current <code>archeo2.db</code> from GitHub.
<code>--db latest</code>	Start from the newest <code>archeo2_new_...db</code> in <code>ingest_work</code> , i.e. carry on from last time.
<code>--db FILE</code>	Start from a particular local file.
<code>--out FILE</code>	Name the output file yourself instead of <code>archeo2_new_<date>.db</code> .

Options:

Flag	Meaning
<code>--dry-run</code>	Do everything that costs nothing (download, list photographs, find duplicates, print the plan and estimate) and then stop.
<code>--replace</code>	When a photographed book is already in the database, regenerate its entry in place instead of skipping it.
<code>--limit N</code>	Generate only the first N entries of the plan, as a trial.
<code>--workers N</code>	Process N books at once (default 3; 4 or 5 is fine).
<code>--yes</code>	Skip the confirmation question.
<code>--summary-model NAME</code>	Model that writes the entries (default <code>claude-opus-5</code>). The writing call is small, so this is cheap whatever you choose.
<code>--research-model NAME</code>	Model that searches the web and gathers the notes (default <code>claude-sonnet-5</code> ; <code>claude-haiku-4-5</code> is the economical choice). Most of the cost is here.
<code>--extract-model NAME</code>	Model for reading photographs (default <code>claude-sonnet-5</code>).
<code>--workdir FOLDER</code>	Use a folder other than <code>ingest_work</code> .
<code>--upload</code>	When finished, push the new file to GitHub (needs <code>GITHUB_TOKEN</code> ; see §7).

Every run that would spend money prints the number of entries, a rough time and cost estimate, and waits for you to press Return before proceeding.

5. Worked examples

Example 1: starting fresh from the published database

You have photographed a batch of new arrivals into a folder called `new_books` and want to add them to what is currently on the website.

```
python3 archeobooks_ingest.py --images new_books --dry-run
```

The dry run downloads the current database, reports how many books it holds, and lists the photographs it found; it costs nothing and is the easiest way to confirm that the setup is right. Then:

```
python3 archeobooks_ingest.py --images new_books
```

The program reads each photograph and prints what it saw, merges the readings into books, and prints a plan:

```
Downloading current database from GitHub ...
Database has 920 books.
Found 8 photographs in new_books.
Reading photographs ...
  IMG_4412.jpg: [copyright page] 'Southwestern Indian Tribes' by Tom Bahti (high)
  IMG_4413.jpg: [title page] 'The Mythic World of the Zuni' by Frank Hamilton Cushing, Barton Wright (editor and
illustrator) (high)
  ...
Merged into 8 books; written to ingest_work/candidates.json (edit and re-run with --manifest if anything is wrong).

Plan:
SKIP      id   167  The Mythic World of the Zuni  (already present; --replace to overwrite)
ADD       id   921  Southwestern Indian Tribes   by Tom Bahti
ADD       id   922  The Sacred Wisdom of the Native Americans  by Larry J. Zimmerman
  ...
7 entries to generate with claude-opus-5, 3 at a time: roughly 0.1 hours and on the order of $4 (rough estimate).
Proceed? [Y/n]
```

Press Return. Each book takes a minute or two; as each finishes you see a line such as

```
[3/7] added      id   924  The Stars Above Us: Or the Conquest of Superstition  (1957, 212 pp, confidence high)
      note: Publisher, year and page count confirmed from library records; ...
```

and at the end

```
Done. Added 7, replaced 0, skipped 1, failed 0.
New database: ingest_work/archeo2_new_20260908_161506.db
Report:      ingest_work/ingest_report_20260908_161506.md
```

Read the report (§7), then preview the database in the website interface:

```
python3 preview_db.py
```

which opens the newest output file (a particular file can be named instead; see the preview manual).

If a title or author was misread from a photograph, open `ingest_work/candidates.json` in a text editor, correct it, and rerun with `--manifest ingest_work/candidates.json` in place of `--images new_books`.

Example 2: the next batch, carrying on from last time

A week later you have more books. Their photographs can go into the same folder or a new one; here they are added to `new_books`. Because the entries from Example 1 are not yet on GitHub, the program must start from your previous output rather than from the download, which is what `--db latest` does:

```
python3 archeobooks_ingest.py --images new_books --db latest
```

The output shows the difference:

```
Using local database: ingest_work/archeo2_new_20260908_161506.db
Database has 927 books.
Found 12 photographs in new_books.
Reading photographs ...
  IMG_4412.jpg: [copyright page] 'Southwestern Indian Tribes' by Tom Bahti (cached)
  ...
  IMG_4501.jpg: [title page] 'Living the Sky: The Cosmos of the American Indian' by Ray A. Williamson (high)
  ...
Plan:
SKIP      id   921  Southwestern Indian Tribes  (already present; --replace to overwrite)
  ...
ADD       id   928  Living the Sky: The Cosmos of the American Indian  by Ray A. Williamson
```

Photographs seen before are marked "cached" and were not sent anywhere; books already present are skipped; only the new ones are generated, and the output is a new file that contains everything so far. Each run builds on the last in this way until you publish (\$7), after which the published file contains everything and you return to the default download.

If you want to build on an older file rather than the most recent, name it explicitly: `--db`

`ingest_work/archeo2_new_20260908_161506.db`.

Example 3: regenerating entries that already exist

There are two situations. The first is that you photograph a book and discover from the plan that it is already in the database, with an entry you do not trust. Adding `--replace` tells the program to regenerate the entry in place, keeping its id:

```
python3 archeobooks_ingest.py --images new_books --db latest --replace
```

Plan:

```
REPLACE id 167 The Mythic World of the Zuni (was: The Mythic World of the Zuni)
```

Be aware that `--replace` applies to every photographed book that matches an existing entry, so use it on a folder containing just the books you mean to redo.

The second situation is that you know the ids of entries you distrust, perhaps from browsing the website or a previous report, and have no photographs. `--refresh` takes ids and ranges:

```
python3 archeobooks_ingest.py --refresh 167,878 --db latest
python3 archeobooks_ingest.py --refresh 1-50 --db latest
```

The existing title and author are treated as reliable and everything else is verified afresh; the research call is told that the old record came from an earlier AI system and may be wrong. The report shows the new entry, and the previous text remains in the source file if you want to compare.

Example 4: adding the shelf code to the database

The shelf code (`FRA.001`, three letters of the first author's surname and a sequence number) is a column in the database, so that it can be searched on the website and exported for labels. Adding it to a database that does not yet have it costs nothing and takes a few seconds:

```
python3 archeobooks_ingest.py --add-codes
```

```
Downloading current database from GitHub ...
Added the Code column to the database.
Assigned shelf codes to 920 entries (0 already had one).
New database: ingest_work/archeo2_new_20260911_172059.db
Label spreadsheet: ingest_work/labels_all_20260911_172059.csv
```

Codes are assigned in id order, so where several books share a surname they run `SAG.001`, `SAG.002` and so on, and a book that already has a code keeps it: once a label is printed and stuck on a spine, its code never moves, whatever else is later corrected in the row. From then on every new entry is given the next free code as it is written, and every run that adds or replaces entries also writes a `labels_<date>.csv` of just those books, which `make_labels.py` turns into an Avery 5160 sheet.

The website does not show a column it has not been told about, so publishing the code (and the publisher, which the database has always held but the page never displayed) means replacing `index.html` in the repository along with the database.

Example 5: cleaning the whole database

The older pipeline left three kinds of damage: duplicate rows (55 titles appear two or three times, 61 rows in all), summaries whose opening paragraphs were lost so that they begin mid-stream with "Chapter ...", and summaries containing fabricated page-cited quotations. The program can find all three, and the sensible way to proceed is in stages, checking the report at each one.

First, see the extent of it for free:

```
python3 archeobooks_ingest.py --dedupe --refresh suspect --dry-run
```

```
Downloading current database from GitHub ...
Database has 920 books.
Found 55 duplicated titles (61 rows to remove):
  keep id    5 (4162 chars), remove id 794 (6081 chars)   Cosmology
  ...
Queued 261 existing entries for regeneration (suspect).

Plan:
(261 entries to regenerate; list omitted)

Dry run complete; 261 entries would be generated. Nothing was changed.
```

Second, a small trial so you can judge the regenerated entries before committing to hundreds:

```
python3 archeobooks_ingest.py --dedupe --refresh suspect --db latest --limit 10 --research-model claude-haiku-4-5
```

Third, the suspect entries in full. Four at a time, 260 books take a little over an hour and a half; the program prints its own estimate:

```
python3 archeobooks_ingest.py --dedupe --refresh suspect --db latest --workers 4 --research-model claude-haiku-4-5
```

Each finished entry is written into the new file at once and the checkpoint is updated, so you may stop at any time with Ctrl-C:

```
^C
Interrupted. Everything finished so far is saved; run again with --resume to continue.
```

and later, after the usual three lines that start a session,

```
python3 archeobooks_ingest.py --resume
```

```
Resuming: 140 of 261 entries already finished; writing into ingest_work/archeo2_new_20260909_083012.db
```

The final report for a resumed run covers everything, including the entries generated before the interruption, and has a "Duplicates removed" section listing every deleted row with its author so that you can confirm that two genuinely different editions were not merged (the duplicate check matches on title alone; two translations of the *Tao Te Ching*, for instance, will be treated as one book).

Finally, if the suspect entries come out well and you want every summary in the database rewritten under the new rules:

```
python3 archeobooks_ingest.py --refresh all --db latest --workers 4
```

This is the one expensive command: 859 books at roughly a book every ninety seconds, four at a time, is about five and a half hours and, at the time of writing, on the order of \$400 with Opus 5 or twice that with Fable 5.1. It resumes in the same way if interrupted.

6. How an entry is judged suspect

`--refresh suspect` is a filter on the stored text, not a judgment of truthfulness; deciding whether a claim is fabricated requires researching the book, which is the expensive thing the filter exists to ration. An entry is flagged if any one of these holds:

- the summary is empty or nothing but whitespace;
- it begins with "Chapter" or "In Chapter", the signature of the old pipeline's truncation, where the opening paragraph identifying the book was lost and the entry starts mid-stream inside a chapter-by-chapter walkthrough;
- it contains a page citation, matched as "(p. 123", "(pp. 123" or "on page 123". This is the clearest fabrication marker in the database, since a model writing from search results has no access to a book's physical pagination, and an invented page number usually has an invented quotation attached to it;

- it is shorter than 400 characters, too thin to be a real catalogue entry and usually a sign that generation failed partway.

The chapter-by-chapter habit is the deeper pattern behind two of those tests: the old prompt invited a walkthrough of the book's structure, and a model that has not seen the table of contents will confabulate one fluently. The current prompts forbid chapter structures unless the research stage actually found a table of contents, which is why replacements are often shorter and vaguer about organisation, and more trustworthy for it.

What the filter cannot catch is the smooth fabrication: a well-formed entry of decent length, no page citations, quietly wrong about what the book argues. Those are indistinguishable from good entries by text alone, and they are the argument for eventually running `--refresh all`.

7. Reading the report

The report opens with a table, one row per book, giving the action taken (added, replaced, skipped, failed), the database id, the verified title, author, year and page count, the confidence rating, and either the reason for skipping or the verification notes. Below the table each book has its own section with the details read from the photographs, the confidence, the verification notes, the sources consulted, the keywords, the summary as written to the database, and, under a collapsible "Research notes" heading, the notes the research stage produced before the summary was written.

The research notes are the best single test of quality. A summary that stands on a research section full of confirmed facts from publisher and library records deserves trust; a summary whose research section says that little could be verified should be short and cautious, and if it is not, that entry deserves a second look. Confidence "low" is the program's own flag for the same thing.

The report is a Markdown file, which TextEdit opens as readable plain text; Visual Studio Code (free) renders it properly with Cmd+Shift+V, and Typora or Obsidian do the same.

8. Previewing and publishing

`python3 preview_db.py` starts a small web server on your Mac and opens the library page, headed "PREVIEW: `archeo2_new_...`", pointed at the newest output file (or at whichever file you name on the command line). Search and browse as a student would; press Ctrl-C in Terminal to stop. Nothing is uploaded.

To compare versions of the same entries, `compare_entries.py` writes one document holding each book's old and new text side by side:

```
python3 compare_entries.py --ids 42,117,388 --labels "published,new" \
  ingest_work/archeo2_current.db ingest_work/archeo2_new_20260911_103526.db
```

With the ids left out it compares every entry whose summary differs between the files.

For a closer look at the raw table, DB Browser for SQLite (free, sqlitebrowser.org) opens the file directly; the Browse Data tab sorted by id descending shows the newest rows, and individual cells can be edited or rows deleted there if a particular entry needs a manual fix.

When you are satisfied, publishing is a matter of replacing `archeo2.db` in the GitHub repository with the new file, renamed to `archeo2.db`: through GitHub Desktop, or by dragging the file onto the repository page in a browser and committing. The website reads the file afresh on every visit. Once published, future runs can go back to the default download instead of `--db latest`.

The `--upload` flag does the same through the GitHub API. It needs a fine-grained personal access token (GitHub: Settings > Developer settings > Personal access tokens) with *Contents: read and write* on the `archeobooks` repository, set as `export GITHUB_TOKEN="github_pat_..."` in the same way as the API key. Check the resulting commit on GitHub the first time you use it.

9. Troubleshooting

"*ANTHROPIC_API_KEY is not set*": the `export` line has not been run in this Terminal window; run it again.

"*No module named anthropic*": the virtual environment is not active; run `source venv/bin/activate` and check that `(venv)` appears.

"*Image folder not found*": you are in the wrong folder, or the folder name is misspelt; `pwd` shows where you are and `ls`

what is there.

A photograph is read wrongly: correct `ingest_work/candidates.json` and rerun with `--manifest ingest_work/candidates.json`. If the same photo is misread every time, the copyright page is usually a better subject than the cover.

A summary contains text like `cite index="12-3"` : stray citation markup; `python3 archeobooks_ingest.py --scrub ingest_work/archeo2_new_<date>.db` removes it from every entry in that file.

"FAILED" against a book: the API call failed after several retries, usually a temporary network or rate-limit problem; the book is listed in the report and a `--resume` will try it again.

Two output files from one session: each run makes its own `archeo2_new_<date>.db`; the one that matters is the one named in the "New database:" line at the end, which shares its stamp with the report.

10. Cost

Reading a photograph costs a fraction of a cent. A verified entry is two calls with up to six web searches, and what it costs depends almost entirely on which model does the research:

Research model	Writing model	Per entry	300 entries
Opus 5	Opus 5	about \$0.54	about \$160
Sonnet 5	Opus 5 (the default)	about \$0.37	about \$110
Haiku 4.5	Opus 5	about \$0.19	about \$56
Fable 5.1	Fable 5.1	about \$1.02	about \$300

The writing model matters little to the bill and much to the prose, so paying for Opus there is close to free. The research model is where strength shows: deciding what to search for an obscure title, judging whether a catalogue record refers to the same edition, and saying plainly that little could be verified rather than filling the gap. The program prints an estimate before each run and waits for confirmation; actual spending, itemised, appears at console.anthropic.com under Usage and Cost, and receipts for credit purchases under Settings > Billing.

11. Settings inside the program

The first screen of `archeobooks_ingest.py` holds the settings you might want to change: the GitHub location of the database, the two default models, the number of web searches allowed per book, how similar two titles must be to count as the same book, the size to which photographs are shrunk before upload, and `LIBRARY_FOCUS`, the one-sentence description of the library's subject that steers the keywords and the emphasis of every summary. Editing that sentence is the simplest way to change the character of the entries the program writes.